

Lecture 4.2: Constructing Regular Expressions - Part 2

Presenter: Azwad Anjum Islam (AAI)

Scribe: Mujtahid Al-Islam Akon (AKO)

In this lecture, we shall continue learning to construct regular expressions through some more examples.

For the following examples, assume the alphabet set, $\Sigma = \{0, 1\}$

Problem 5

Construct a regular expression for all binary strings that **contain the substring 1011**.

Answer

If we add any binary string before **or** after **1011**, it will give the desired binary string recognized by the language. We already know that any binary string of length 0 or more is $(0|1)^*$.

So, the accepting pattern should be, any binary string 1011 any binary string. Replacing any binary string part with $(0|1)^*$, we get the result, $(0|1)^*1011(0|1)^*$.

Problem 6

Construct a regular expression for all binary strings that **contain the subsequence 1011**.

Answer

From the concept of subsequence, we understand that, any binary string part (mentioned in the previous example) can even appear in each side of each symbol of the string, **1011**.

So, the required format should be,

"any binary string 1 any binary string 0 any binary string 1 any binary string 1 any binary string".

So, the result is $(0|1)^* 1 (0|1)^* 0 (0|1)^* 1 (0|1)^* 1 (0|1)^*$

N.B. The following solution is also correct. Can you find out why?

$(0^*)1(1^*)0(0^*)1(0^*)1(0|1)^*$

Problem 7

Construct a regular expression for all binary strings **that do NOT contain consecutive 1s**.

Answer

You have already known that $(0|1)^*$ generates all possible strings with **0 & 1**

i.e. $\{\epsilon, 0, 1, 00, 01, 10, \text{11}, 000, 001, 010, \text{011}, \dots\}$

1. Notice that the above RegEx would work well for this problem except the strings marked **red** are problematic as they contain more than **one** consecutive 1's. To solve this issue, what you can do is replacing the **1** with **10**.
After applying this idea, we get,
 $(0|10)^* = \{\epsilon, 0, 10, 00, 010, 100, \text{1010}, 000, 0010, 0100, \text{01010}, \dots\}$
Notice that, **no** consecutive 1s is there anymore.
2. Now, notice again that $(0|10)^*$ could not cover some strings those should have been covered to complete the answer. It could not cover the strings where **1 is present at the end**. For example, $\{1, 01, 001, 101, \dots\}$, all of them are valid strings and should be accepted but are missed by $(0|10)^*$. So, we need to add a new term cover them and we can do so by appending a **1** with $(0|10)^*$ i.e. $(0|10)^*1$.

Combining the two parts, we get,

$$(0|10)^* \mid (0|10)^*1$$
$$\Rightarrow (0|10)^*(\epsilon|1)$$

Another correct solution is $(\epsilon|1)(0|01)^*$. Find out why by yourself.

Note: Always verify your regular expressions with some strings from the language. Don't miss the corner cases.

Problem 8

Construct a regular expression for all binary strings **that do NOT contain consecutive 0s**.

Answer

$(1|01)^*(\epsilon|0)$ (Similar to **Problem 7**).

Problem 9

Construct a regular expression for all binary strings in which **the number of 0's is divisible by 3**.

Answer

As long as you keep getting 1s, everything is fine. But whenever a **0** appears, we need to wait for 2 more 0s to accept the string. So far, we get $(1|000)^*$. But these 0s do not need to come consecutively. So, we put any number of 1s around each of them.

Applying all these, finally we get $(1|0 1^* 0 1^* 0)^*$.

Problem 10

Construct a regular expression for **all mobile phone numbers of Bangladesh**.

For example: +8801515654784, 01911720127

Answer

Consider the following cases-

- The strings can contain either +88 or nothing. So, we get $(+88|\epsilon)$ or in short $(+88)?$.
- Then the string must be followed by 01.
- Then we need to add the operator codes. For example
 - GP: 3,7
 - Robi: 6, 8
 - Banglalink: 9
 - Teletalk: 5So, we need to append $(3|5|6|7|8|9)$
- At last, we need to add last 8 digits. They can be any decimal digits from 0 to 9. So, we can again use a shorthand notation $[0 - 9]^8$.

Now concatenating all of them we get our desired RE, $(+88)? 01(3|5|6|7|8|9)[0|9]^8$

To test by yourself how regular expressions work in real life as well as to play with it, you can explore <https://regexr.com>.